

Chapter 5

Database

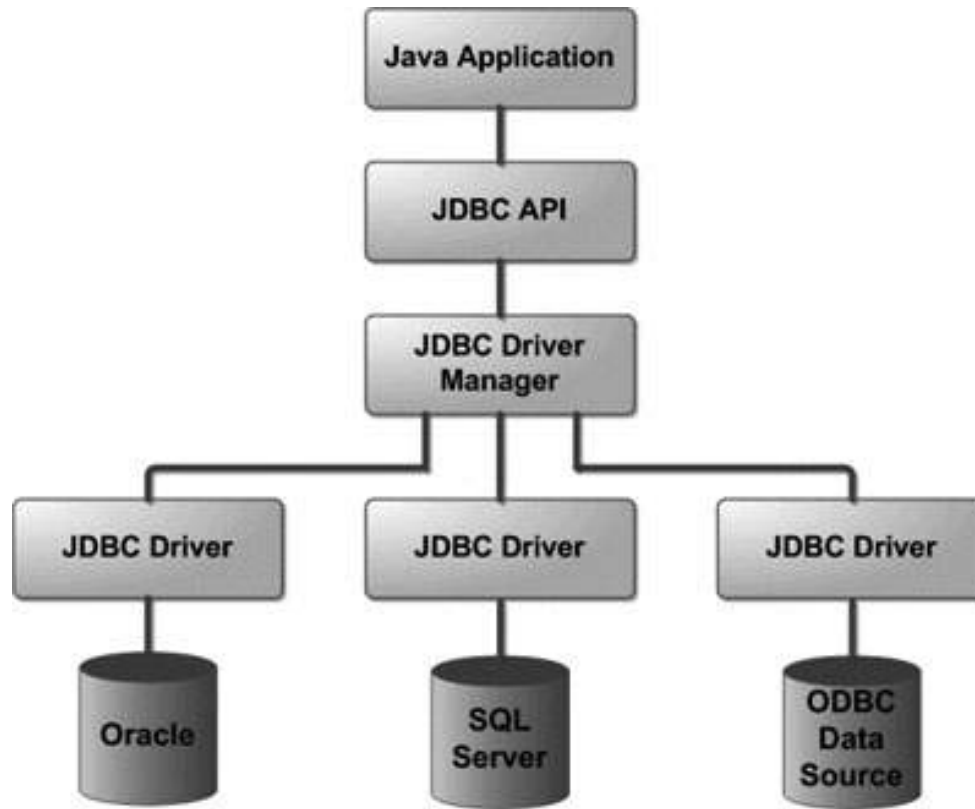
1 Introduction

- JDBC is the API used by Java applications to interact with a database
- There are too many databases out there, and internal format of databases are different
 - ◆ for example, the internal format of an Oracle database will be different from that of an MySQL database
- Mediating software that will allow JDBC to communicate with a specific database is necessary.
 - ◆ Such software is referred to as a driver.
- Java provides an API for SQL access along with a driver manager
 - ◆ The Driver Manager provides a basic service for managing a set of JDBC drivers

1 Introduction

- Drivers are usually supplied either by the database vendors themselves or by third parties.
- There would then be a simple mechanism for registering third-party drivers with the driver manager.
- The JDBC API defines the Java interfaces and classes that programmers use to connect to databases and send queries.
- A JDBC driver implements these interfaces and classes for a particular DBMS vendor.
- A Java program that uses the JDBC API loads the specified driver for a particular DBMS before it actually connects to a database.
- The JDBC DriverManager class then sends all JDBC API calls to the loaded driver.

1 Introduction



2. Types of JDBC Drivers

- A JDBC driver is a software component enabling a Java application to interact with a database.
- To connect with individual databases, JDBC requires drivers for each database.
- There are four types of JDBC drivers
 - ◆ Type 1 driver - JDBC-ODBC bridge driver
 - ➔ translates JDBC to ODBC and relies on an ODBC driver to communicate with the database.
 - ◆ Type 2 driver - Native-API driver
 - ➔ Uses the client-side libraries of the database. The driver converts JDBC method calls into native calls of the database API.

2. Types of JDBC Drivers

- ◆ Type 3 driver - Network-Protocol driver (Middleware driver)
 - makes use of a middle tier between the calling program and the database.
 - The middle-tier converts JDBC calls into a vendor-specific database protocol.
- ◆ Type 4 driver - Database-Protocol driver (Pure Java driver) or thin driver.
 - converts JDBC calls directly into a vendor-specific database protocol.
 - They install inside the Java virtual machine of the client.
 - provides better performance than the type 1 and type 2 drivers.
 - Unlike the type 3 drivers, it does not need associated software to work.

3 JDBC Configuration

- You need a database program for which a JDBC driver is available.
- Obtain the JAR file in which the driver for your database is located.
- Include the driver JAR file on the class path
 - ◆ The classpath tells the JDK tools and applications where to find third-party and user-defined classes that are not extensions or part of the Java platform.
 - ➔ The CLASSPATH helps the Java Virtual Machine or JVM know where to look for resources.
 - ◆

4. Connecting to a Database

- To connect to a database first you need to register the driver.
 - ✓ Find out the name of the JDBC driver classes, then
 - ✓ Use the `forName()` method of `Class` class to register the driver class.
 - ➔ e.g
 - `Class.forName("oracle.jdbc.driver.OracleDriver");` *//oracle driver*
 - `Class.forName("com.mysql.jdbc.Driver");` *//MySQL JDBC driver*
 - `Class.forName(org.apache.derby.jdbc.ClientDriver)` *//java DB*
 - ✓ Since JDBC 4.0, explicitly registering the driver is optional.
 - ➔ Include the drivers class's location in the program's classpath.
 - ➔ JDBC driver manager can detect and load the driver automatically.

4. Connecting to a Database

- After loading the driver, establish connection using the `DriverManager.getConnection()` method.

```
Connection con = DriverManager.getConnection(String url, String user,  
                                             String password)
```

- ◆ URL is an address that points to your database.

```
jdbc:mysql://hostname/databaseName
```

```
jdbc:oracle:thin:@hostName:portNumber:databaseName
```

```
jdbc:derby://hostName:portNumber/databaseName
```

- The driver manager iterates through the registered drivers to find a driver that can use the subprotocol specified in the database URL.
- The `getConnection` method returns a `Connection` object.

5. Working with JDBC Statements

- Once a connection is obtained we can interact with the database.
- The JDBC Statement, CallableStatement, and PreparedStatement interfaces define the methods and properties that enable you to send SQL commands and receive data from your database.
- Statement interface is useful when you are using static SQL statements at runtime.
 - ◆ Cannot accept parameters.
 - ◆ Created using the Connection object's `createStatement( )` method
`Statement stmt = conn.createStatement()`
 - ◆ Used to execute an SQL statement with one of its three execute methods.

5. Working with JDBC Statements

- ◆ **boolean execute (String SQL)**: Returns true if a ResultSet object can be retrieved.
 - Used to execute SQL DDL statements.
- ◆ **int executeUpdate (String SQL)** – Returns the number of rows affected by the execution of the SQL statement.
 - Used to execute SQL statements for which you expect to get a number of rows affected - for example, an INSERT, UPDATE, or DELETE statement.
- ◆ **ResultSet executeQuery (String SQL)** – Returns a ResultSet object.
 - Use this method when you expect to get a result set, as you would with a SELECT statement.

5. Working with JDBC Statements

➤ The PreparedStatement Interface

- ◆ Extends the Statement interface,
- ◆ Gives you the flexibility of supplying arguments dynamically.
- ◆ we can prepare a query with parameters and use it many times.
- ◆ Created like:

```
PreparedStatement pstmt = null;  
try {  
    String SQL = "Update Employees SET age = ? WHERE id = ?";  
    pstmt = conn.prepareStatement(SQL);  
}  
catch (SQLException e) { }
```

5. Working with JDBC Statements

- ◆ ? represents parameters, and known as parameter marker.
- ◆ You must supply values for every parameter before executing the SQL statement.
- ◆ The setXXX() methods bind values to the parameters.
 - XXX represents the Java data type of the value you wish to bind to the input parameter.
- ◆ Each parameter marker is referred by its ordinal position.
 - The first marker represents position 1, the next position 2, and so forth.
- ◆ All of the Statement object's methods for interacting with the database (a) execute(), (b) executeQuery(), and (c) executeUpdate() also work with the PreparedStatement object.

5. Working with JDBC Statements

```
PreparedStatement pstmt = null;  
try {  
    String SQL = "Update Employees SET age = ? WHERE id = ?";  
    pstmt = conn.prepareStatement(SQL);  
    pstmt.setInt(1, 30);  
    pstmt.setString(2, "R/24");  
    pstmt.executeUpdate();  
}  
catch (SQLException e) { }  
finally{  
    pstmt.close();  
}
```

5. Working with JDBC Statements

➤ The CallableStatement Interface

- ◆ Used to execute a call to a database stored procedure.
- ◆ Stored procedures are set of SQL statements stored in a database.
 - ➔ programs accessing that database can invoke them.
- ◆ CallableStatements can receive arguments and output parameters in which a stored procedure can place return values.

CallableStatement cstmt =

con.prepareCall("{call Procedure_name(?, ?)}");

6. ResultSet Interface

- ResultSet is a table of data generated by executing database queries.
- is retrieved by calling `executeQuery()` on an object of Statement or PreparedStatement interfaces.

```
PreparedStatement pstmt = con.prepareStatement("select * from  
employees");  
ResultSet rs = pstmt.executeQuery();
```

- The ResultSet object maintains a cursor that points to the current row of the result set.
- Use `next()` on a ResultSet to iterate through the records.

6. ResultSet Interface

- Use the getX(ColName) methods to fetch the values from the database columns, where
 - ◆ X is the datatype of the column.
 - ◆ ColName is database column names.

```
while(rs.next()) {  
    String name = rs.getString("name");  
    Integer Id = rs.getInt("idnumber");  
    Double gpa = rs.getDouble("gpa");  
}
```

- ◆ The index number of the column can be used with the getX() methods instead of the column name.

6. ResultSet Interface

```
String name = rs.getString(1);  
int Id = rs.getInt(2);  
double salary = rs.getDouble(3);
```

- Information about the column properties and types in a ResultSet can be retrieved using getMetaData() method

```
ResultSetMetaData metaData = rs.getMetaData();
```

- Use methods on metadata object to retrieve properties of each column.
- To get the number of columns of the result set:

```
int columnCount = metaData.getColumnCount();
```

6. ResultSet Interface

- When we obtain a ResultSet, the position of the cursor is before the first row.
- By default, the ResultSet moves only in the forward direction.
- Use a scrollable ResultSet for other navigation options.
 - ◆ TYPE_FORWARD_ONLY – the default option, in which the cursor moves from start to end
 - ◆ TYPE_SCROLL_INSENSITIVE – cursor can move in both forward and backward directions; if there are changes to the underlying data while moving through the dataset, they are ignored.
 - ◆ TYPE_SCROLL_SENSITIVE – The dataset immediately reflects any changes to the underlying data

6. ResultSet Interface

- Not all databases support all the ResultSet types.
 - ◆ We can check if the type is supported by using the `supportsResultSetType()` method on `DatabaseMetaData` object:

```
DatabaseMetaData dbmd = con.getMetaData();  
boolean isSupported = dbmd.supportsResultSetType(  
    ResultSet.TYPE_SCROLL_INSENSITIVE);
```

- To get a scrollable ResultSet, we need to pass some additional parameters while preparing the Statement.

```
PreparedStatement pstmt = con.prepareStatement(  
    "select * from student", ResultSet.TYPE_SCROLL_INSENSITIVE,  
    ResultSet.CONCUR_UPDATABLE);
```

6. ResultSet Interface

```
PreparedStatement pstmt = con.prepareStatement( "select * from student",  
    ResultSet.TYPE_SCROLL_SENSITIVE,  
    ResultSet.CONCUR_UPDATABLE);  
ResultSet rs = pstmt.executeQuery();  
while (rs.next()) {} // iterate through the results from first to last  
rs.beforeFirst(); // jumps back to the starting point, before the first row  
rs.afterLast(); // jumps to the end of resultset  
rs.first(); // navigates to the first row  
rs.last(); // goes to the last row  
rs.absolute(2); //jumps to 2nd row  
rs.relative(-1); // jumps to the previous row  
rs.relative(2); // jumps forward two rows  
while (rs.previous()) {} // iterates from current row to the first row
```

6. ResultSet Interface

- The concurrency mode indicates if our ResultSet can update the data.
 - ◆ The CONCUR_READ_ONLY option is the default
- If we need to update the data in our ResultSet, then the CONCUR_UPDATABLE option should be used.
- To check if a ResultSet support concurrency modes use the supportsResultSetConcurrency() method:

```
DatabaseMetaData dbmd = con.getMetaData();  
boolean isSupported = dbmd.supportsResultSetConcurrency(  
    ResultSet.TYPE_SCROLL_SENSITIVE,  
    ResultSet.CONCUR_UPDATABLE
```

6. ResultSet Interface

- To update a row of a ResultSet use updateX() methods.

```
rs.updateFloat("gpa", 3.88f);
```

```
rs.updateRow()
```

- ◆ call updateRow() to save the updates to the database:

- To insert a new row, do the following:

```
rs.moveToInsertRow(); //move the cursor to insert a new row
```

```
rs.updateString("name", "Markos"); //set name
```

```
rs.updateString("idnumber", "R/30"); //set id number
```

```
rs.updateFloat("gpa", 3.23f); //set gpa
```

```
rs.insertRow(); //insert the new row into the database
```

```
rs.moveToCurrentRow(); //take the cursor back to the row we  
were at before we started inserting a new row
```

6. ResultSet Interface

- To delete a row in updatable ResultSet.

rs.**absolute**(2); *//navigate to the row we want to delete.*

rs.**deleteRow**(); *// delete throw*



7. The RowSet Interface

- The RowSet interface extends the ResultSet interface
- It configures the database connection and prepares query statements automatically.
- The following are subinterfaces of RowSet.
 - ◆ A CachedRowSet allows disconnected operation.
 - ➔ Caches the data of a ResultSet in memory and disconnects from the database.
 - ◆ A WebRowSet is a cached row set that can be saved to an XML file.
 - ◆ The FilteredRowSet and JoinRowSet interfaces support lightweight operations on row sets that are equivalent to SQL SELECT and JOIN operations.
 - ◆ A JdbcRowSet allows connected operations.

7. The RowSet Interface

- RowSet is scrollable and updatable by default

```
JdbcRowSet rowSet =  
    RowSetProvider.newFactory().createJdbcRowSet();  
rowSet.setUrl("jdbc:derby:myDb");  
rowSet.setUsername(USERNAME);  
rowSet.setPassword(PASSWORD);  
rowSet.setCommand("SELECT * FROM student");  
rowSet.execute();
```

8. Transactions

- You can group a set of statements to form a transaction.
- The transaction can be
 - ◆ committed when all has gone well—or,
 - ◆ rolled back as if none of the statements had been issued if an error has occurred in one of them.
- The major reason for grouping statements into transactions is database integrity.
- If you group update statements into a transaction, the transaction
 - ◆ either succeeds in its entirety and can be committed, or
 - ◆ it fails somewhere in the middle. and rollback.

8. Transactions

- By default, a database connection is in autocommit mode
 - ◆ each SQL statement is committed to the database as soon as it is executed.
- To turn off autocommit mode and then group two or more statements into transaction

```
conn.setAutoCommit(false);
```
- After the auto-commit mode is disabled, no SQL statements are committed until you call the method `commit` explicitly.

8. Transactions

```
con.setAutoCommit(false);  
stmt.setString(1, "R/23");  
stmt.setFloat(2, 4.0f);  
stmt.executeUpdate();
```

```
stmt.setString(1, "R/25");  
stmt.setFloat(2, 4.0f);  
stmt.executeUpdate();  
con.commit();
```